

---

# Lab W05: An Adventuring Party, Part 1

CIS 3300, Intermediate Object-Oriented Programming

---

## A note on how this assignment was developed

This assignment was drafted with AI help (mostly for brainstorming new problems to solve and formatting with LaTeX). It should be accurate, but something might still be wrong. Because this is a new assignment, you are also helping me debug it, and I will take that into consideration when I review your work. That said, helping me find rough edges is not a substitute for doing the assignment; each problem below is solvable, but you may need to make assumptions in your attempt. If you find an error or something unclear, please tell me; that helps later versions, though finding errors is not the point of this assignment.

---

## 1 Scenario

You are building a small piece of software for a tabletop role-playing game. Before a group can go adventuring, someone has to represent each character in code: a name, a class (Wizard, Fighter, and so on), and six ability scores that describe what the character is good at.

This is Part 1 of a two-week assignment. This week is about getting the basic shape of three classes right: fields, one constructor each, and plain getters and setters. Next week you will come back to this same code and expand it further. Nothing here, this week or next, involves interfaces.

## 2 Learning Objectives

By the end of this part, you should be able to:

1. **Organize** a multi-class Eclipse project using packages and a dedicated folder for data files.
2. **Write** a class with private fields, one constructor, and plain getters and setters.
3. **Decide** which fields need a setter and which should only ever be read, not changed.
4. **Read** structured data from a text file and use it to construct a collection of objects.
5. **Keep** a collection inside one class fully encapsulated, exposing it only through specific methods rather than returning it directly.
6. **Write** a program's output to a file rather than to the console.
7. **Design** loop and method logic that reads clearly from top to bottom, with a stopping condition that is obvious from the loop itself.

## 3 Project Setup

1. Create a new Eclipse project named `labw05`. Right click the `src` folder, choose `New > Package`, and name it `adventure`. Create `Adventurer.java`, `Party.java`, and `Quest.java` in that package as you write them; each file's first line should read `package adventure;`.

- 
2. Right click the project name, choose New > Folder, and name it `files`. Place all the data files you plan to use, `adventurers.txt`, and optionally `adventurers_sample2.txt` and `adventurers_sample3.txt`, in this folder. That is where your program will look for them when you run it from Eclipse. Your program will also save its output file to this same folder.
  3. Right click the `src` folder again, choose New > Package, and name it `app`. Create `PartyDemo.java` in that package; its first line should read `package app;`.

## 4 Commenting Requirements

Every file you submit must follow the syllabus's Code Commenting Requirements section, written so the `javadoc` tool could generate documentation from your code without changes:

- A general comment block at the top of every file, including this assignment's title, a version number (start at 1.0), your name as the author, and at least one sentence describing the class.
- A comment on every method describing what it does, its parameters, and what it returns.
- A comment on every static constant field. Part 1 does not have any yet; Part 2 will.

See the syllabus's Code Commenting Requirements section for the full detail, including the referenced Core Java example listing.

## 5 What You Will Build

Three classes:

- **Adventurer** – one character, with a name, a class, and six ability scores.
- **Party** – a group of **Adventurer** objects.
- **Quest** – a job for the party to undertake, with a difficulty rating and a gold reward.

You will also write a small program that reads a data file and uses it to build a starting party, using file I/O from earlier in the semester.

## 6 Class Specifications

### 6.1 Adventurer

Fields (all private): `name` and `characterClass` (both `String`), and six `int` ability scores: `strength`, `dexterity`, `constitution`, `intelligence`, `wisdom`, and `charisma`.

**One constructor:** `Adventurer(String name, String characterClass, int strength, int dexterity, int constitution, int intelligence, int wisdom, int charisma)`. Set `name` and `characterClass` directly. Set the six ability scores by calling your six setters (see below) rather than assigning the fields directly.

Give every field a getter. Give the six ability scores setters as well; `name` and `characterClass` do not need setters. A setter should simply assign the value it is given:

```
1 public void setStrength(int strength) {  
2     this.strength = strength;  
3 }
```

Write the other five ability score setters the same way.

---

## 6.2 Party

Fields (all private): a `String` called `partyName`, and an `ArrayList<Adventurer>` called `members`.

**One constructor:** `Party()`, taking no parameters. Inside it, set `partyName` to any default name you like (for example, "Unnamed Party"), and instantiate an empty `ArrayList` for `members`.

Methods:

- `public void addMember(Adventurer adventurer)` – adds an adventurer to the party.
- `public int getMemberCount()` – returns how many adventurers are in the party.
- `public Adventurer getMember(int index)` – returns the adventurer at that position.
- `public Adventurer getMember(String name)` – returns the adventurer whose name matches, ignoring case, by searching through the party. If no adventurer has that name, return `null`. This is a second method named `getMember`; Java tells the two apart by their parameter type, which is called overloading.
- `public String getPartyName()` and `public void setPartyName(String partyName)` – a plain getter and setter.

Do *not* write a method that returns the `ArrayList` itself. The only ways in or out of the list should be `addMember`, `getMemberCount`, and `getMember`.

## 6.3 Quest

Fields (all private): `questName (String)`, `difficultyRating (int)`, and `rewardGold (int)`.

**One constructor:** `Quest(String questName, int difficultyRating, int rewardGold)`. Set `questName` and `difficultyRating` directly. Set `rewardGold` by calling `setRewardGold` (same reasoning as `Adventurer` above).

Give all three fields getters. Give `rewardGold` a plain setter. `questName` and `difficultyRating` do not need setters.

## 7 The Data File

Your program reads a file named `adventurers.txt` from the `files` folder described in Project Setup, one adventurer per line, with each value separated by a single space, in this order: name, class, strength, dexterity, constitution, intelligence, wisdom, charisma.

```
Elandra Wizard 8 14 12 17 13 10
Grom Fighter 17 12 16 8 10 9
Fenwick Rogue 10 18 12 13 14 8
Brynn Cleric 13 10 15 9 17 12
```

## 8 Additional Sample Data

Once your program works against `adventurers.txt`, try it against each of these two files as well, to make sure your program works for a party of any size, not just a party of exactly four. Save whichever one you are testing with in your `files` folder (renaming it, or changing the file name in your code, either is fine). No sample output is given for these two; check that your results look reasonable on your own.

`adventurers_sample2.txt`:

---

```
Thistle Ranger 12 16 11 10 15 9
Orin Bard 9 14 12 11 8 17
Sable Paladin 16 10 15 8 12 14
```

adventurers\_sample3.txt:

```
Quinn Druid 10 13 14 12 17 9
Marrow Warlock 8 12 13 16 10 15
Ivy Sorcerer 9 14 11 13 10 18
Dorian Monk 15 17 13 10 12 8
Wren Artificer 11 12 14 17 13 9
```

## 9 Write Your Own Driver

Create a class called `PartyDemo` with a `main` method. This is the one piece of code you are writing entirely yourself this week; next week's assignment gives you a finished version of it so you can focus on the classes instead.

*Before starting, review the lecture and lecture notes covering file I/O. Reading `adventurers.txt` should follow the same pattern we used there.*

`PartyDemo` must:

- Instantiate a `Party` object and store it in a variable named `party`.
- Use a `Scanner` to read `files/adventurers.txt` and extract the eight values for each adventurer, following the same approach used in the file I/O lecture and notes. Close this `Scanner` when you are finished with it.
- Construct an `Adventurer` from each adventurer's values and add it to `party`.
- Use a `PrintWriter` to write your output to `files/party_report.txt`.
- Use `printf` when writing each adventurer's line, so the six ability scores line up in columns.
- Create one `Quest` with any values you like, and include it in the report.
- Close the `PrintWriter` when you are finished writing.

Here is the shape of the program, in pseudocode, to work from:

```
create an empty Party named party

open "files/adventurers.txt" for reading (fileScanner)
while fileScanner has more data
    read one adventurer's name, characterClass, and six ability scores
    create an Adventurer from those values
    add the Adventurer to party
close fileScanner

open "files/party_report.txt" for writing (reportWriter)
write party's name and member count to reportWriter
for each adventurer in party
```

---

```
    printf the adventurer's name, class, and six raw scores to reportWriter
create a Quest with sample values
printf the quest's name, difficulty, and reward to reportWriter
close reportWriter
```

## 10 Sample Output

Running your program against `adventurers.txt` should produce a `files/party_report.txt` containing the same information as below, organized similarly, with the six scores lined up in columns. Your output does not need to match this character for character, and if you chose a different default party name in your `Party` constructor, that line will show your name instead. For help producing aligned columns like these, see the `printf` guide in the File I/O lectures folder.

Unnamed Party has 4 members.

|         |         |     |    |     |    |     |    |     |    |     |    |     |    |
|---------|---------|-----|----|-----|----|-----|----|-----|----|-----|----|-----|----|
| Elandra | Wizard  | STR | 8  | DEX | 14 | CON | 12 | INT | 17 | WIS | 13 | CHA | 10 |
| Grom    | Fighter | STR | 17 | DEX | 12 | CON | 16 | INT | 8  | WIS | 10 | CHA | 9  |
| Fenwick | Rogue   | STR | 10 | DEX | 18 | CON | 12 | INT | 13 | WIS | 14 | CHA | 8  |
| Brynn   | Cleric  | STR | 13 | DEX | 10 | CON | 15 | INT | 9  | WIS | 17 | CHA | 12 |

Quest: Clear the Coppertop Mine  
Difficulty: 4  
Reward: 200 gold

## 11 Suggested Order

1. Review the file I/O lecture and notes before you start.
2. Write `Adventurer`, including its constructor and all eight getters and setters.
3. Write `Party` and `Quest`.
4. Using the specifications and pseudocode above, write `PartyDemo` yourself.
5. Run it against `adventurers.txt` and compare `files/party_report.txt` to the sample output.
6. Try it against one or both of the additional sample files.

## 12 Grading

This lab is graded using the checklist below. Each item is graded as met or not met, there is no partial credit within a single item, but each item's point value counts toward your total score.

**Table 1:** Submission and setup, 1 point each

| Item                                                                                                                     | Points |
|--------------------------------------------------------------------------------------------------------------------------|--------|
| Submitted as a reasonable attempt                                                                                        | 1      |
| Adventurer.java, Party.java, and Quest.java compile and run in Eclipse without errors, declared in the adventure package | 1      |
| PartyDemo.java compiles and runs in Eclipse without errors, declared in the app package                                  | 1      |
| Project pushed to GitHub, following the walkthrough video                                                                | 1      |

**Table 2:** Code quality, 1 point each

| Item                                                   | Points |
|--------------------------------------------------------|--------|
| Consistent naming conventions used throughout          | 1      |
| Consistent formatting and indentation throughout       | 1      |
| No unrequested extra code or output                    | 1      |
| Adventurer.java has required documentation             | 1      |
| Party.java has required documentation                  | 1      |
| Quest.java has required documentation                  | 1      |
| PartyDemo.java has required documentation              | 1      |
| Loop and method logic reads clearly from top to bottom | 1      |

**Table 3:** Adventurer

| Item                                                                                        | Points |
|---------------------------------------------------------------------------------------------|--------|
| Constructor sets name, characterClass, and the six ability scores to the given values       | 1      |
| Constructor sets the six ability scores through its setters, not by direct field assignment | 1      |
| getName returns name                                                                        | 1      |
| getCharacterClass returns characterClass                                                    | 1      |
| getStrength returns strength                                                                | 1      |
| setStrength correctly assigns strength                                                      | 1      |
| getDexterity returns dexterity                                                              | 1      |
| setDexterity correctly assigns dexterity                                                    | 1      |
| getConstitution returns constitution                                                        | 1      |
| setConstitution correctly assigns constitution                                              | 1      |
| getIntelligence returns intelligence                                                        | 1      |
| setIntelligence correctly assigns intelligence                                              | 1      |
| getWisdom returns wisdom                                                                    | 1      |
| setWisdom correctly assigns wisdom                                                          | 1      |
| getCharisma returns charisma                                                                | 1      |
| setCharisma correctly assigns charisma                                                      | 1      |

---

**Table 4: Party**

| Item                                                                                          | Points |
|-----------------------------------------------------------------------------------------------|--------|
| Constructor takes no parameters and correctly initializes partyName and an empty members list | 2      |
| addMember correctly adds an adventurer to the party                                           | 1      |
| getMemberCount returns the correct number of members                                          | 1      |
| getMember returns the adventurer at the given position                                        | 1      |
| getMember correctly finds the adventurer with a given name, ignoring case                     | 2      |
| getPartyName returns partyName                                                                | 1      |
| setPartyName correctly assigns partyName                                                      | 1      |
| No method exposes the ArrayList directly                                                      | 1      |

**Table 5: Quest**

| Item                                                                             | Points |
|----------------------------------------------------------------------------------|--------|
| Constructor sets questName, difficultyRating, and rewardGold to the given values | 1      |
| Constructor sets rewardGold through its setter, not by direct field assignment   | 1      |
| getQuestName returns questName                                                   | 1      |
| getDifficultyRating returns difficultyRating                                     | 1      |
| getRewardGold returns rewardGold                                                 | 1      |
| setRewardGold correctly assigns rewardGold                                       | 1      |

**Table 6: Reading and reporting**

| Item                                                            | Points |
|-----------------------------------------------------------------|--------|
| adventurers.txt is read and used to build a Party               | 2      |
| Report includes the party info, every adventurer, and one Quest | 1      |
| Six ability scores are lined up in columns using printf         | 1      |
| Scanner is closed                                               | 1      |
| PrintWriter is closed                                           | 1      |

**Total: 50 points.**

## 13 Submission

Submit your entire labw05 Eclipse project to your course GitHub repository, per the syllabus's submission guidelines, not just the individual `.java` files. Your repository should reflect the same structure you built in Eclipse: a `src` folder containing the `adventure` package (`Adventurer.java`, `Party.java`, `Quest.java`) and the `app` package (`PartyDemo.java`), plus the `files` folder containing `adventurers.txt`. It is fine either way whether `files/party_report.txt` ends up in your submission or not, since it is generated automatically whenever your program runs. Keep this project where you can find it; next week's assignment starts from it.